

從 Reward 到 Advantage：強化學習演算法的數學演進史

The Mathematical Evolution of RL Algorithms

Part 1: 起源與數學困境 (The Origin & The Problem)

1.1 強化學習的核心目標 (The Objective)

在 RL 中，我們的終極目標是最大化 **期望累積回報 (Expected Cumulative Reward)**：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

我們希望找到一組參數 θ (Policy Network)，使得這個目標函數最大化。最直觀的方法是 **Gradient Ascent**。

根據 **Policy Gradient Theorem**：

$$\nabla J(\theta) = \mathbb{E}_{\tau} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

- $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ ：**方向**。告訴參數怎麼動可以增加這個動作的機率。
- G_t (Return)：**力道**。這個動作好不好？好就推大力一點，不好就反向推。

1.2 原始方法的致命傷：High Variance

最原始的演算法 **REINFORCE (Monte Carlo Policy Gradient)** 直接使用整條軌跡的實際回報 G_t 。

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{T-t} r_T$$

問題所在：

G_t 是一個隨機變數，其隨機性來自兩個源頭：

- 環境的隨機性 (Transition Probability)**：同樣動作可能導致不同後果。
- 策略的隨機性 (Stochastic Policy)**：Policy 本身就是機率分佈。

金融類比：這就像你只根據單一次的歷史股價路徑 (One Sample Path) 來估計未來的期望報酬。

- $Var(G_t)$ 隨著時間 T 增長而累積（甚至是指數級增長）。
- 後果**：梯度估計充滿雜訊 (Noisy Gradient)，模型訓練像是在隨機漫步 (Random Walk)，難以收斂。

Part 2: 引入控制變數 (Variance Reduction)

2.1 第一步改進：引入 Baseline

為了降低變異數，我們引入一個不依賴於動作 a_t 的基線 $b(s_t)$ 。

$$\nabla J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t))]$$

為什麼這樣有效？

數學上可以證明引入 Baseline 不會改變梯度的期望值 (Unbiased)，但能大幅降低 變異數 (Variance Reduction)。

- $\mathbb{E}[\nabla \log \pi \cdot b(s)] = 0$ (這是數學恆等式)。
- **直觀解釋**：如果所有 G_t 都是正的 (例如 [100, 110])，梯度一直往正向推，很不穩。減去平均值後變成 [-5, +5]，方向更明確。

最佳 Baseline：通常設為狀態價值函數 $V(s_t) \approx \mathbb{E}[G_t | s_t]$ 。

2.2 第二步改進：Critic 的介入 (Bias-Variance Trade-off)

即便減去 Baseline， G_t 本身還是 Monte Carlo Sample，變異數依然很大。我們改用 **Bootstrapping** 的方式，用估計值來取代真實值。

我們定義 **Advantage Function**：

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

但在 Actor-Critic 架構中，我們通常近似為：

$$A(s_t, a_t) \approx r_t + \gamma V(s_{t+1}) - V(s_t)$$

(這就是 TD Error δ_t)

權衡 (Trade-off):

1. **Monte Carlo (G_t)**: Unbiased, High Variance. (相信真實發生的事)
2. **TD (1-step)**: High Bias, Low Variance. (相信模型的估計)
 - Bias 來自於我們用的 $V(s)$ 是一個估計器 (Neural Network)，一開始是錯的。

2.3 最終解方：GAE (Generalized Advantage Estimation)

為了在 Bias 和 Variance 之間找到甜蜜點，我們使用 GAE：

$$A_t^{GAE}(\gamma, \lambda) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

- $\lambda = 0$: TD(0)。只看一步。變異最小，偏差最大。
- $\lambda = 1$: Monte Carlo。看無限步。變異最大，偏差最小。

結論：這是目前處理 RL Reward 訊號的**黃金標準**。大多數現代演算法 (PPO, TRPO) 都使用 GAE 來計算 Advantage。

Part 3: 演算法的演進史 (The Algorithm Evolution)

這一段我們從模型架構的角度，看如何一步步走到今天。

3.1 史前時代：Q-Table & Q-Learning

- **機制**：維護一個巨大的表格 $Q(s, a)$ ，記錄每個狀態做每個動作的價值。
- **更新**： $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
- **數學限制**：
 - **維度災難 (Curse of Dimensionality)**：狀態空間如果是連續的（如股價、機器人關節角度），表格會無限大。
 - 無法處理沒見過的狀態。

3.2 深度學習時代：DQN (Deep Q-Network)

- **突破**：用神經網路 $Q_\theta(s, a)$ 來擬合那個巨大的表格。
- **機制**：
 1. **Experience Replay**：打破資料的時間相關性 (Correlation)。
 2. **Target Network**：固定目標值，避免自己追逐自己 (Stationary Target)。
- **問題**：
 - 它是 **Value-Based**。要得到策略，必須取 $\arg \max_a Q(s, a)$ 。
 - 這在連續動作空間 (Continuous Action Space) 是不可行的（要解一個優化問題才能得到動作）。

3.3 策略梯度：Actor-Critic (A2C/A3C)

- **突破**：直接學習策略 $\pi_\theta(a|s)$ (Actor)，同時學習價值 $V_\phi(s)$ (Critic) 來幫忙算 Advantage。
- **解決了什麼**：可以處理連續動作空間。
- **新問題**：步長 (Step Size) 太難調。
 - Learning Rate 太小 $\rightarrow$ 收斂極慢。
 - Learning Rate 太大 $\rightarrow$ 策略更新過頭，掉進懸崖 (Performance Collapse)，且因為資料分佈改變 (Off-policy issue)，再也救不回來。

3.4 數學約束：TRPO (Trust Region Policy Optimization)

- **核心思想**：我們希望每次更新策略時，新的策略 π_{new} 不要離舊的策略 π_{old} 太遠。
- **數學定義**：在優化目標函數時，加入 **KL Divergence** 的硬約束 (Hard Constraint)。

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_{\theta}}{\pi_{old}} A_t \right] \quad \text{subject to} \quad KL[\pi_{old}(\cdot|s), \pi_{\theta}(\cdot|s)] \leq \delta$$

- **缺點**：求解這個帶約束的優化問題需要計算 **Hessian Matrix** (二階導數) 的反矩陣，計算量是 $O(N^3)$ ，在深層網路中極度昂貴。

3.5 集大成者：PPO (Proximal Policy Optimization)

- **突破**：用一階導數 (First-order) 近似 TRPO 的二階優化。
- **核心機制 (Clip)**：不需解複雜的 KL 約束，直接在 Objective Function 裡動手腳：

$$L^{CLIP}(\theta) = \mathbb{E} [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

其中 $r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{old}(a|s)}$ 是機率比率。

- **直觀解釋：**
 - 如果 Advantage 是正的（好動作）：盡量提高機率，但不要超過舊機率的 $(1 + \epsilon)$ 倍。
 - 如果 Advantage 是負的（壞動作）：盡量降低機率，但不要低於舊機率的 $(1 - \epsilon)$ 倍。
- **優點：**簡單、運算快、樣本效率高 (Sample Efficient)。這也是 ChatGPT (RLHF) 背後的核心演算法。

Part 4: 總結 (Summary)

演算法	核心機制	解決的問題	遺留的問題
REINFORCE	G_t (Monte Carlo)	最直觀的策略優化	High Variance, 收斂慢
A2C	$Q - V$ (Advantage)	降低 Variance	更新步長敏感，容易訓練崩潰
TRPO	KL Divergence Constraint	保證單調進步 (Monotonic Improvement)	計算量太大 (Hessian Matrix)
PPO	Clipped Surrogate Objective	模擬 TRPO 但計算便宜	目前工業界的首選

對熊熊的 Takeaway:

從 Financial Engineering 的角度看，RL 的演進史其實就是一部「如何在充滿雜訊的市場訊號中，尋找穩定 Alpha」的數學史。

- **Baseline** = 去除大盤影響 (Beta)。
- **Advantage** = 超額報酬 (Alpha)。
- **GAE** = 在動量 (Trend) 與均值回歸 (Reversion) 之間調整估計窗格。
- **PPO Clip** = 風險控管 (Risk Management)，防止單次下注過大導致破產。